

PaperProof Contracts Observability and Read APIs

This document describes how the current contracts can be monitored from off-chain systems and which additional event/read surfaces are still worth considering in future iterations.

The goal is to help:

- frontend developers
- indexer developers
- analytics builders
- governance observers
- community members monitoring protocol activity

1. Overview

The current protocol already exposes a meaningful set of:

- events
- public getters
- protocol root objects

That means off-chain systems can already track the most important lifecycle changes without relying on hidden state.

Because the system has grown more sophisticated, this document also distinguishes between:

- observability that is already implemented
- observability that is still only recommended for a later version

2. Current Event Surface

Publishing events

The publishing package currently emits events for the most important lifecycle milestones:

- `CodeReserved`
- `PaperFinalized`
- `PaperVersionAdded`
- `PaperOwnerTransferred`
- `CommentsTreeBound`
- `PaperUiStatusChanged`
- `StorageExtended`
- `ConfigUpdated`

These are enough to monitor:

- paper reservation flow
- first publication flow
- version growth
- owner handoff
- comments–tree binding
- official UI moderation/display–state changes
- storage–extension activity
- key registry configuration changes

Comments events

The comments package already has relatively strong observability:

- `TreeCreatedEvent`
- `CommentAddedEvent`
- `TreeStatusChangedEvent`
- `CommentStatusChangedEvent`
- `TreeOwnerTransferredEvent`
- `PaperLikedEvent`
- `PaperUnlikedEvent`

These are enough to monitor:

- discussion tree creation
- comment growth
- paper-like activity
- moderation/status-marker changes
- discussion-governance ownership changes

Governance core and voting events

The governance core now emits:

- `OperatorNominatedEvent`
- `OperatorTransferAcceptedEvent`
- `OperatorTransferCancelledEvent`
- `FeeRecipientChangedEvent`
- `PublishingFeeLevelChangedEvent`
- `CommentsFeeLevelChangedEvent`
- `UpgradeAuthorityChangedEvent`
- `ManagedUpgradeCapRegisteredEvent`
- `ManagedUpgradeAuthorizedEvent`
- `ManagedUpgradeCommittedEvent`
- `GovernanceVaultMigratedEvent`

These are enough to monitor:

- operator handoff lifecycle
- protocol fee-policy changes
- official fee recipient rotation
- official upgrade-authority changes
- managed `UpgradeCap` custody registration
- governed upgrade authorization/commit flow
- governance-vault migration activity

The governance voting layer already emits:

- `GovernanceConfigCreatedEvent`
- `ProposalCreatedEvent`

- `VoteCastEvent`
- `ProposalFinalizedEvent`
- `ProposalExecutedEvent`
- `VoteClaimedEvent`
- `GovernanceConfigMigratedEvent`
- `ProposalMigratedEvent`
- `ProposalCreationPausedChangedEvent`
- `ProposerThresholdChangedEvent`

These are enough to monitor:

- governance initialization
- proposal creation cadence
- voting participation
- proposal outcomes
- execution of executable proposals
- locked-vote fund reclamation
- governance-config migration
- proposal migration
- governance pause-state changes
- proposer-threshold changes

3. Current Read API Surface

Publishing public getters

The publishing package already exposes useful read functions for:

- registry and record versioning
- registry paused state
- registry limits as a grouped read
- paper code
- paper epoch and sequence
- record status

- UI status
- paper owner
- current version
- version count
- version IDs
- bound `comments_tree_id`
- record timestamps as a grouped read
- record lookup by paper code

This gives frontends and indexers direct access to the main paper registry state.

Comments public getters

The comments package already exposes a large read surface, including:

- tree identity and version
- creator and current owner
- registry ID and paper object binding
- root comment ID
- total comments
- next comment ID
- tree status
- comment limits
- like count
- whether a given address has liked
- comment existence
- comment borrow helpers
- comment author/depth/mode/status
- blob and preview fields
- current protocol constants for modes/statuses

This is already strong and is sufficient for most frontend rendering and analytics work.

Governance public getters

The governance package already exposes read functions for:

- vault version
- governance authority
- upgrade authority
- active operator
- active operator epoch
- pending operator transfer details
- fee recipient
- fee levels
- current fee amounts

The voting layer also exposes getters for:

- governance config version
- total supply
- proposer threshold
- proposal-creation pause state
- next proposal ID
- active proposal ID
- proposal object ID lookup
- proposal status and execution state
- yes/no votes
- yes/no locked values
- proposal epoch window
- whether a given address has voted
- per-address vote power
- whether a given address can reclaim locked tokens
- whether a proposal is currently executable
- governance constants

4. Current Strengths

The current observability design is already good in three ways:

1. Major lifecycle actions are visible

The most important publication, discussion, and governance actions all emit events.

2. Root protocol objects are queryable

The architecture exposes enough public getters for frontend and indexer work without forcing everything into event-only interpretation.

3. Versioned-upgrade preparation is inspectable

The public version getters make it possible to observe migration progress and detect whether core objects are still on the expected protocol version.

5. Remaining Future Event Additions

The most important observability gaps have now been filled in governance core and publishing. The remaining future additions are comparatively optional.

Comments events worth considering

Comments is already in relatively good shape.

Potential future additions are minor, for example:

- explicit `CommentReplyAddedEvent` separate from generic comment-added flow
- explicit `PaperLikeStateResetEvent` only if future like semantics expand

These are optional, not urgent.

6. Remaining Future Read Functions

The current getter surface is already strong, but a few additions could still improve off-chain ergonomics.

Governance getters worth considering

- getter for managed–upgrade package list or managed–upgrade object IDs
- getter exposing whether governance is in an operationally frozen/upgrade mode if such a state is introduced later

Proposal/voting getters worth considering

- getter for whether a proposal is currently executable
- getter for whether a given voter can reclaim locked voting funds

These would simplify wallet–side UX.

7. What Does Not Need Immediate Expansion

Not every possible action needs a dedicated event or getter immediately.

The following areas are now already reasonably observable:

- comment tree growth
- proposal creation and conclusion
- vote casting
- vote claims
- publication and version lifecycle
- operator lifecycle
- fee–policy changes
- upgrade–authority changes
- managed–upgrade authorization/commit flow
- proposal/config migration activity

So the next most meaningful additions are no longer basic governance–control visibility, but rather a few ergonomic convenience reads and any future comments–specific refinements.

8. Practical Guidance for Off–Chain Builders

Frontends

Use:

- getters for canonical current state
- events for timeline/history

Do not rely on events alone when rendering protocol–critical state such as:

- current fee levels
- current operator
- current proposal status
- current paper owner

Indexers

Index:

- publishing lifecycle events
- comment lifecycle events
- governance proposal/vote events
- root shared object IDs and version numbers

Also persist:

- package IDs by release
- current root object versions
- active governance config values

Community watchers

For now, the most meaningful things to monitor are:

- new proposals
- proposal execution
- operator changes
- fee changes
- package upgrade preparation and execution
- migration activity

9. Summary

The current `paperproof-contracts` system now has a strong observability and read-API base across `publishing`, `comments`, `governance`, and `governance_voting`.

The highest-value additions that were previously only recommended are now implemented:

- governance-core operator events
- fee-change events
- upgrade-authority and managed-upgrade events
- governance/config/proposal migration events
- publishing-side `CommentsTree` binding and UI-status events
- convenience getters for protocol-critical publishing and voting state

The remaining future work is mostly incremental rather than foundational.